

Examen réparti 2

Master 1 SAR

UE 4I401

L. Arantes, J. Sopena, P. Sens

Janvier 2024

2 heures – **Aucun document autorisé**

Tout appareil électronique interdit

Le barème sur 20 points est donné à titre indicatif

Exercice 1 – Système de fichiers (5,5 points)

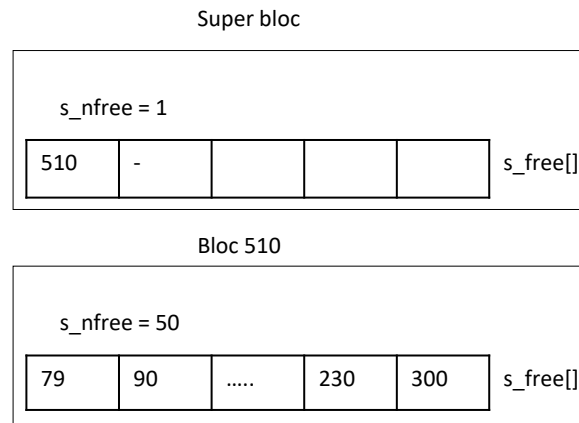
Les *inodes* ont une structure équivalente à celle vue en TD. Les inodes sont numérotées à partir de 1, ont une taille de 32 octets et sont stockées à partir du bloc numéro 2. Les blocs disque ont une taille de 512 octets.

Le répertoire courant (u.u_cdir) correspond à l'inode numéro 10. Nous considérons les *inodes* et les blocs suivants :

i_number = 10		i_number = 20		i_number = 40		i_number = 41	
i_addr[]		i_addr[]		i_addr[]		i_size = 10 i_addr[]	
0	500	0	800	0	900	0	1000
1	600	1		1		1	0
2	700	2		2		2	0
3		3		3		3	
4		4		4		4	
5		5		5		5	
6		6		6		6	
7		7		7		7	
8		8		8		8	
9		9		9		9	
10		10		10		10	
11		11		11		11	
12		12		12		12	

Bloc 600	Bloc 700	Bloc 800	Bloc 900	Bloc 1000
"rep1" 20	"rep2" 40	"fich1" 41	"fich2" 43 "fich3" 41	"examen2024"

Le superbloc a initialement la configuration suivante :



Question 1 $\frac{1}{2}$ point

Faites un schéma de l'arborescence du système de fichiers en représentant les éventuelles liens physiques.

Solution:

```

__rep1__fich1
|__rep2__fich2
|__fich3

```

fich1 et fich3 sont des liens physiques vers la même inode.

Soit le code utilisateur suivant :

```

1  int fd1,fd2;
2  char buf[16];
3  fd1 = open("./rep1/fich1", 0_RDWR);
4  lseek(fd1, 2*512, SEEK_SET);
5  if(fork()==0) {
6      write(fd1, "1234", 4);
7      fd2 = open("./rep2/fich3", 0_RDWR);
8      write(fd2, "abc", 3);
9      if (fork()==0) {
10         close(fd1);
11         read(fd2, buf, 3);
12         buf[3] = 0;
13         printf("%s\n", buf);
14         close(fd2);
15         exit(0);
16     }
17     else {
18         wait(NULL);
19         read(fd2, buf, 3);
20         buf[3] = 0;
21         printf("%s\n", buf);
22         close(fd1); close(fd2);
23         exit(0);
24     }
25 }
26 wait(NULL);
27 close(fd1);
28 exit(0);

```

Question 2 $\frac{1}{2}$ point

Quel est l'affichage produit par le printf des lignes ?? et ?? (justifiez) ?

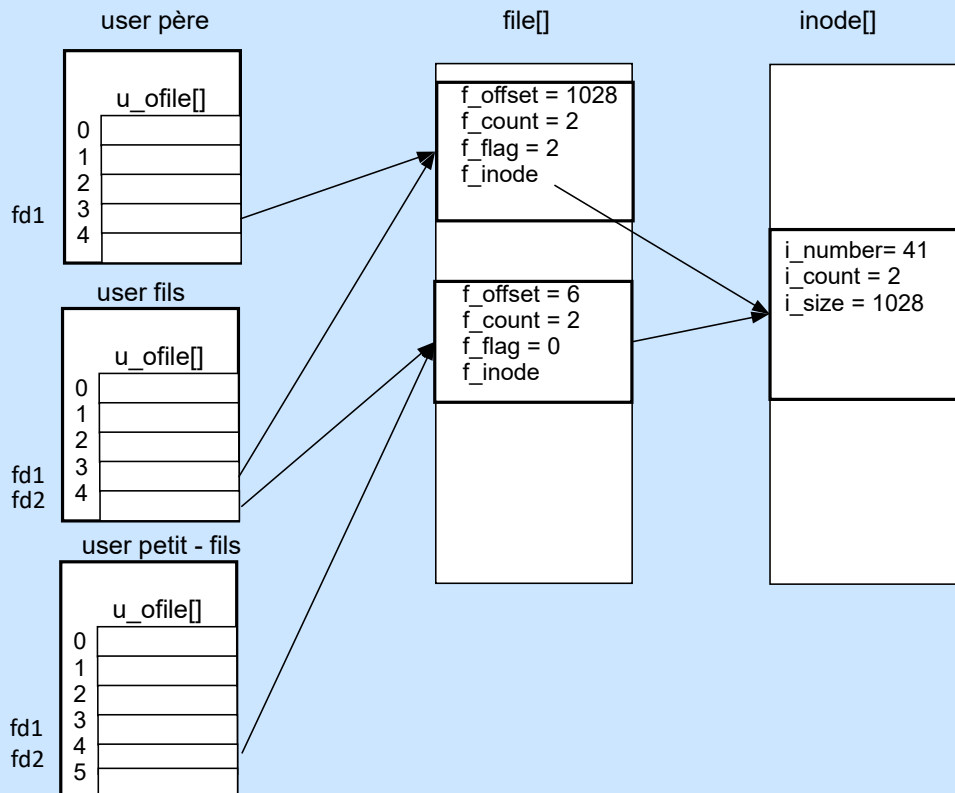
Solution:

Ligne ?? : "men"

Ligne ?? : "202"

Question 3 1½ points

Donnez l'état juste après la ligne ?? des tables de descripteurs de fichiers, fichiers ouverts, et inodes (faites un schéma, pensez à représenter notamment les champs *f_count*, *f_offset*, *f_flag*, *i_count*, *i_size* et *i_number*).

Solution:**Question 4** 1½ points

Avant l'exécution de ce code, nous faisons l'hypothèse que seule inode du répertoire courant (inode numéro 10) est présente dans la table *inode[]*. Donnez dans l'ordre la séquence des blocs et des inodes accédés par la ligne ??.

Solution:

open -namei
 inode 10 en mémoire
 accès bloc 500, 600
 accès bloc inode 3 ($2 + (20-1)/16$)
 accès inode 20
 accès bloc 800
 accès bloc inode 4 ($2 + (41-2)/16$)
 accès inode 41

Question 5 1½ points

Quels sont les blocs accédés par la ligne ?? . Donnez l'état du superbloc juste après la ligne ??.

Solution:

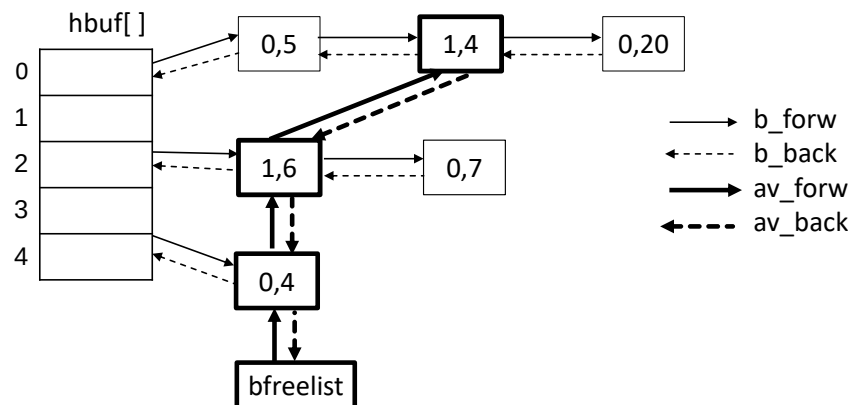
Accès en écriture à un nouveau bloc => allocation/accès bloc 510

Le SB devient vide, le contenu de 510 est copié dans le SB

- s_nfree = 50
- sfree = [79, 90, ..., 230, 300]

Exercice 2 – Buffer cache (5 points)

Nous considérons un buffer cache de **6 tampons**. La figure suivante donne l'état du buffer cache. Dans chaque buffer est indiqué le doublet $\langle dev, bloc \rangle$ qui correspond au numéro de device et de bloc associé au buffer. On suppose qu'aucun buffer n'a initialement le flag DEL_WRI.



Question 1 1/2 point

Les tampons du buffer caches sont-ils alloués dans les espaces utilisateur ou système ?

Solution:

Système

Question 2 1 point

Quels sont les buffers qui ne sont pas en cours d'utilisation ? Dans quel ordre ont eu lieu les entrées/sorties ont eu lieu sur ces buffers.

Solution:

Blocs <0,4>, <1,6> et <1,4> avec les E/S faites dans cet ordre.

Question 3 1 1/2 points

Le noyau exécute la séquence suivante :

```
1 struct buf *bp;
2 bp = bread(0,12);
3 brelse(bp);
4 bp = bread(1,6);
5 brelse(bp);
6 bp = bread(2,4);
7 brelse(bp);
```

Donnez l'état de la bfreelist (la liste des blocs qui la compose de la tête vers la queue) après les lignes 3, 5 et 6 ainsi que le nombre d'entrées/sorties physiques générées par cette séquence (les codes de brelse, bread et bdwrite sont rappelés en annexe).

Solution:

Init : $\langle 0, 4 \rangle$ $\langle 1, 6 \rangle$ $\langle 1, 4 \rangle$

ligne 3 $\langle 0, 12 \rangle$: $\langle 1, 6 \rangle$ $\langle 1, 4 \rangle$ $\langle 0, 12 \rangle$, 1 E/S

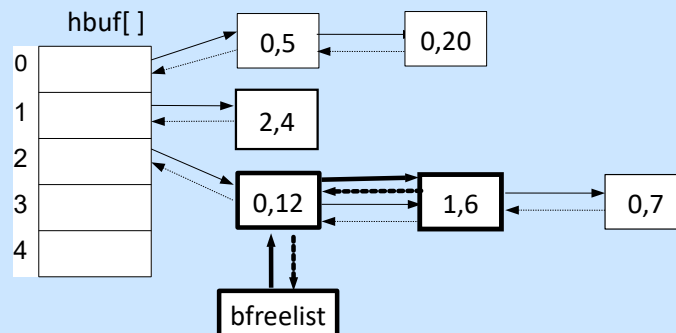
ligne 5 $\langle 1, 6 \rangle$: $\langle 1, 4 \rangle$ $\langle 0, 12 \rangle$ $\langle 1, 6 \rangle$

ligne 6 $\langle 2, 4 \rangle$: $\langle 0, 12 \rangle$ $\langle 1, 6 \rangle$, 1 E/S

Total : 2 E/S

Question 4 1 point

Faites un schéma complet du buffer cache (hbuf, chainage des buffers et de la bfreelist) après la ligne 6.

Solution:

Barème : 30 % répartition des buffers, 70% chainages

Question 5 1 point

On suppose maintenant qu'initialement le bloc $\langle 0, 4 \rangle$ a le flag DEL_WRI positionné. Quel est l'état de la bfreelist après la ligne 3 en supposant que la durée d'une entrées/sorties en écriture est plus courte qu'une entrée/sortie en lecture ?

Solution:

ligne 3 $\langle 0, 12 \rangle$: $\langle 1, 4 \rangle$ $\langle 0, 4 \rangle$ $\langle 0, 12 \rangle$

Exercice 3 – Swap (3 points)

On considère le swaper du TD dont le code est rappelé en annexe.

On considère que la mémoire comporte **10 unités** et la table `proc` ne comporte que 5 processus. Voici l'état de cette table juste après l'exécution de la fonction `realtime` où `p_size` est exprimé en nombre d'unités :

p_pid	p_flag	p_stat	p_size	p_time	p_wchan	p_pri
0	SSYS SLOAD	SSLEEP	2	200	&runout	PSWP
1	SSYS SLOAD	SRUN	2	190		PZERO - 1
100	SLOAD	SSLEEP	2	3	&toto	PZERO - 2
101	SLOAD	SSLEEP	2	5	&toto	PZERO - 1
102	SLOAD	SSLEEP	2	4	&toto	PZERO + 1
103	0	SSLEEP	3	1		

On fait les hypothèses suivantes :

- Aucun `fork` ni `exit`, ni `wakeup(&toto)` ne sera appelé par la suite.
- Le temps d'attente de fin d'entrée/sortie est considéré négligeable par rapport à une seconde.
- On ne prendra pas en compte les segments texte des processus.

Question 1 3 points

Le processus 103 passe à l'état `SRUN`, donnez l'évolution de la table jusqu'à ce qu'il soit chargé en mémoire. Si besoin vous pouvez considérer un ou plusieurs appels à `realtime`. Pour simplifier, vous pouvez lister uniquement la liste des champs qui changent.

Solution:

Le passage à l'état `SRUN` entraîne :

1. switch vers le swapper immédiatement car il est endormi sur `runout` (`wakeup` dans le `sleep`) et qu'il a la plus forte priorité.
2. le swapper choisi 103 pour le sortir du swap car ce dernier est `SRUN` mais pas `SLOAD`
3. impossible de charger 103 car la mémoire est pleine, il choisit donc de swapper 102 car à l'état `sleep` à une priorité interruptible ($\Rightarrow$ I/O pour swapper 102).
4. une fois l'I/O terminée , le swapper se réveille dans `xswap` : on enlève le `SLOAD` de 102, son `p_time` passe à 0
5. il rechoisit 103 $\rightarrow$ toujours pas de place en mémoire et aucun processus à évincer (tous `SSYS` ou bien avec à l'état `SSLEEP` mais à une priorité non-interruptible), 103 étant depuis moins de 3 sec. sur le swap, **le swapper s'endort donc sur `runin`**
6. après 2 sec., 103 est depuis 3 sec. en mémoire, cette fois ci 101 peut être choisi même s'il est non interruptible car il est depuis le plus longtemps en mémoire.
7. 101 est déchargé, les `p_time` de 101 et 103 passent à 0.

On a donc

p_pid	p_flag	p_stat	p_size	p_time	p_wchan	p_pri
0	SSYS SLOAD	SSLEEP	2	202	&runout	PSWP
1	SSYS SLOAD	SRUN	2	192		PZERO - 1
100	SLOAD	SSLEEP	2	5	&toto	PZERO - 2
101	0	SSLEEP	2	0	&toto	PZERO - 1
102	0	SSLEEP	2	2	&toto	PZERO + 1
103	SLOAD	SRUN	3	0		

Exercice 4 – Synchronisation et appel système (7,5 points)

On souhaite réaliser un nouveau mécanisme de verrouillage de fichiers implémentant un protocole lecteur/écrivain. Pour cela on veut implémenter les 4 appels systèmes suivants :

- `lock_read(int fd)` appelée par les processus lecteurs, qui bloque l'appelant si le fichier correspondant au descripteur `fd` est déjà ouvert en écriture.
- `lock_write(int fd)` appelée par les processus écrivains, qui bloque l'appelant si le fichier correspondant au descripteur `fd` est utilisé par un autre processus (i.e., un autre processus a fait directement un `open` ou a hérité de l'ouverture d'un parent).
- `unlock_read(int fd)` qui signale la fin de lecteur sur `fd` et débloque éventuellement un écrivain bloqué dans `lock_write`
- `unlock_write(int fd)` qui signale la fin d'écriture et débloque éventuellement un processus bloqué dans `lock_write` ou `lock_read`

La figure suivante illustre un scénario de verrouillage.

Processus Lecteur 1	Processus Lecteur 2	Processus Ecrivain
<pre>int fd=open("/tmp/fich", O_RDONLY); lock_read(fd); /*accès en lecture */ read(fd,buf, 10); unlock_read(fd);</pre>	<pre>int fd=open("/tmp/fich", O_RDONLY); lock_read(fd); /*accès en lecture */ read(fd,buf, 5); unlock_read(fd);</pre>	<pre>int fd=open("/tmp/fich", O_WRONLY); lock_write(fd); /*BLOQUE */ /*DEBLOQUE */ /*accès en ecriture */ write(fd,buf, 5); unlock_write(fd);</pre>

On suppose que les processus lecteurs ou écrivains **n'ouvrent qu'une seule fois le fichier** avant d'appeler les fonctions `lock_read` ou `lock_write`.

Question 1 1½ points

Donnez le code de la fonction interne `int check_write(struct inode *ip)` qui retourne 0 si aucun processus n'utilise le fichier désigné par `ip` en mode écriture (i.e., avec le flag `FWRITE` dans `f_flag`) et 1 sinon.

Solution:

```
int check_write(struct inode *ip) {
    struct file *pf;
    for (pf=&file[0]; pf<&file[NFILE]; pf++)
        if (pf->f-count > 0 && pf->f_inode == ip && pf->f_flag&FWRITE ) {
            return 1;
        }
    return 0;
}
```

Question 2 1½ points

Donnez le code dans le noyau de l'appel système `sys_lock_read()` correspondant à

`int lock_read(int fd)`. L'appel doit retourner 0 et retourne l'erreur EBADF si `fd` ne correspond pas à un descripteur de fichier valide ou à un fichier non ouvert en lecture.

Tant qu'un accès write est en cours, l'appel doit se bloquer à une priorité non interruptible en appelant `sleep`, le processus positionne alors préalablement le flag `PREAD` dans le champs `i_flag` de l'inode.

Solution:

```
sys_lock_read() {
    struct file *fp=u.u_ofile[u.u_arg[0]];

    if (fp==NULL || !fp->f_flag&FREAD) {
        u.u_error = EBADF;
        return;
    }

    while (check_write(fp->f_inode)) {
        fp->f_inode->i_flag |= PREAD;
        sleep((caddr_t) fp->f_inode+1, PZERO -1);
        fp->f_inode->i_flag &= ~PREAD;
    }
    u.u_ar[R0] = 0;
}
```

Question 3 1½ points

Donnez le code dans le noyau de l'appel système `sys_lock_write()` correspondant à `int lock_write(int fd)`. L'appel doit retourner 0 et retourne l'erreur EBADF si `fd` ne correspond pas à un descripteur de fichier valide ou à un fichier non ouvert en écriture.

Tant qu'un autre processus utilise le fichier, l'appelant doit se bloquer à une priorité non interruptible et positionner préalablement le flag `PWRITE` dans le champs `i_flag` de l'inode.

Solution:

```
sys_lock_write() {
    struct file *fp=u.u_ofile[u.u_arg[0]];

    if (fp==NULL || !fp->f_flag&FWRITE) {
        u.u_error = EBADF;
        return;
    }

    while (fp->f_count > 1 || fp->f_inode->i_count > 1) {
        fp->f_inode->i_flag |= PWRITE;
        sleep((caddr_t) fp->f_inode+1, PZERO -1);
        fp->f_inode->i_flag &= ~PWRITE;
    }
    u.u_ar[R0] = 0;
}
```

Question 4 1½ points

Donnez le code dans le noyau de l'appel système `sys_unlock_read()` correspondant à `int unlock_read(int fd)`. L'appel doit retourner 0 (on fait l'hypothèse que le descripteur `fd` est correct).

Attention un processus écrivain bloqué ne doit être réveillé qu'à l'appel de `unlock_read` par le dernier lecteur.

Solution:

```
sys_unlock_read() {
    struct file *fp=u.u_ofile[u.u_arg[0]], *pf;
    int count=0;

    if (fp->f_inode&PWRITE) {
        for (pf=&file[0]; pf<&file[NFILE]; pf++)
            if (pf->f_count > 0 && pf->f_inode == fp->f_inode && pf->f_flag&
                FREAD)
                count += pf->f_count;

        if (count == 1 )
            /* Dernier lecteur */
            wakeup((caddr_t) fp->f_inode+1);
    }
    u.u_ar[R0] = 0;
}
```

Question 5 1½ points

Donnez le code dans le noyau de l'appel système `sys_unlock_write()` correspondant à `int unlock_write(int fd)`. L'appel doit retourner 0 (on fait l'hypothèse que le descripteur `fd` est correct).

Solution:

```
sys_unlock_write() {
    struct file *fp=u.u_ofile[u.u_arg[0]];
    if (fp->f_inode&(PWRITE|PREAD) )
        wakeup((caddr_t) fp->f_inode+1);
    u.u_ar[R0] = 0;
}
```

Annexe

```
#define NADDR 13

struct inode
{
    char        i_flag;
    char        i_count; /* reference count */
    dev_t       i_dev;   /* device where inode resides */
    ino_t       i_number; /* i number, 1-to-1 with device address */
    unsigned short i_mode;
    short       i_nlink; /* directory entries */
    short       i_uid;   /* owner */
    short       i_gid;   /* group of owner */
    off_t       i_size;  /* size of file */
    struct {
        daddr_t    i_addr[NADDR]; /* if normal file/directory */
        daddr_t    i_lastr;       /* last logical block read (for read-ahead) */
    };
};

struct inode inode[NINODE]; /* The inode table itself */

/*
 * One file structure is allocated
 * for each open/creat/pipe call.
 */
struct file
{
    char        f_flag;
    cnt_t       f_count; /* reference count */
    struct inode *f_inode; /* pointer to inode structure */
    long        f_offset; /* read/write character index */
} file[NFILE];

/* flags */
#define FOPEN      (-1)
#define FREAD      0x0001
#define FWRITE     0x0002
#define FPIPE      0x0004

/* open parameters */
#define O_RDONLY   0
#define O_WRONLY   1
#define O_RDWR     2
```

```

/*
 * The main loop of the scheduling (swapping) process.
 * The basic idea is:
 *   see if anyone wants to be swapped in;
 *   swap out processes until there is room;
 *   swap him in;
 *   repeat.
 * The runout flag is set whenever someone is swapped out.
 * Sched sleeps on it awaiting work.
 *
 * Sched sleeps on runin whenever it cannot find enough
 * memory (by swapping out or otherwise) to fit the
 * selected swapped process. It is awakened when the
 * memory situation changes and in any case once per second.
 */

```

```

sched()
{
    register struct proc *rp, *p;
    register struct text *xp;
    register int a, n;

    /*
     * find user to swap in;
     * of users ready, select one out longest
     */

loop:
    spl(CLINHB);
    n = -1;
    for (rp = &proc[0]; rp < &proc[NPROC]; rp++)
        if (rp->p_stat==SRUN && (rp->p_flag&SLOAD) == 0 &&
            rp->p_time > n) {
            p = rp;
            n = rp->p_time;
        }
    /*
     * If there is no one there, wait.
     */
    if (n == -1) {
        runout++;
        sleep((caddr_t)&runout, PSWP);
        goto loop;
    }
    spl(NORMAL);

    /*
     * See if there is memory for that process;
     */

    rp = p;
    a = rp->p_size;
    if((xp=rp->p_textp) != NULL)
        if(xp->x_ccount == 0)
            a += xp->x_size;
    if((a=malloc(coremap, a)) != NULL)
        goto found2;

    /*

```

```

* none found.
* look around for easy memory.
* Select the largest of those sleeping
* at bad priority; if none, select the oldest.
*/

spl(CLINHB);
for (rp = &proc[0]; rp < &proc[NPROC]; rp++)
    if ((rp->p_flag&(SSYS|SLOCK|SLOAD))==SLOAD &&
        (rp->p_stat==SSLEEP && rp->p_pri >= PZERO) || rp->p_stat==SSTOP))
        goto found1;
if(n < 3)
    goto sloop;
n = -1;
for (rp = &proc[0]; rp < &proc[NPROC]; rp++)
    if ((rp->p_flag&(SSYS|SLOCK|SLOAD))==SLOAD &&
        (rp->p_stat==SRUN || rp->p_stat==SSLEEP) &&
        rp->p_time > n) {
        p = rp;
        n = rp->p_time;
    }
if(n < 2)
    goto sloop;
rp = p;

/*
* swap user out
*/
found1:
spl(NORMAL);
rp->p_flag &= ~SLOAD;
xswap(rp, 1, 0);
goto loop;

/*
* swap user in
*/
found2:
rp=p;
if((xp=rp->p_textp) != NULL) {
    if(xp->x_ccount == 0) {
        if(swap(xp->x_daddr, a, xp->x_size, B_READ))
            goto swaper;
        xp->x_caddr = a;
        a += xp->x_size;
    }
    xp->x_ccount++;
}
if(swap(rp->p_addr, a, rp->p_size, B_READ))
    goto swaper;
mfree(swapmap, (rp->p_size+7)/8, rp->p_addr);
rp->p_addr = a;
rp->p_flag |= SLOAD;
rp->p_time = 0;
goto loop;

sloop:
runin++;
sleep((caddr_t)&runin, PSWP);
goto loop;

```

```
swaper:
    panic("swap_error");
}
```

```

/*
 * Read in (if necessary) the block and return a buffer pointer.
 */
struct buf *
bread(dev, blkno)
    dev_t dev;
    daddr_t blkno;
{
    register struct buf *bp;

    bp = getblk(dev, blkno);
    if (bp->b_flags & B_DONE)
        return(bp);
    bp->b_flags |= B_READ;
    bp->b_bcount = BSIZE;
    (*bdevsw[bmajor(dev)].d_strategy)(bp);
    iowait(bp);
    return(bp);
}

/*
 * release the buffer, with no I/O implied.
 */
brelse(bp)
    register struct buf *bp;
{
    register struct buf **backp;
    register s;

    if (bp->b_flags & B_WANTED)
        wakeup((caddr_t)bp);
    if (bfreelist.b_flags & B_WANTED) {
        bfreelist.b_flags &= ~B_WANTED;
        wakeup((caddr_t)&bfreelist);
    }
    if (bp->b_flags & B_ERROR) {
        bp->b_flags &= ~(B_ERROR | B_DELWRI);
        bp->b_error = 0;
    }

    s = spl();
    spl(BDINHB);
    backp = &bfreelist.av_back;
    (*backp)->av_forw = bp;
    bp->av_back = *backp;
    *backp = bp;
    bp->av_forw = &bfreelist;
    bp->b_flags &= ~(B_WANTED | B_BUSY | B_ASYNC);
    bfreelist.b_bcount++;
    spl(s);
}

```